

# Fraudulent E-Commerce Transaction Detection via Multiple Machine Learning Approaches

Fabrizio Ficarella  
Giovanni Michele Iuliani  
Gianluca Lascaro  
Filippo Morbidelli  
Claudio Pio Pignatelli  
Elio Maria Trincia  
Group C, Machine Learning I Project  
Italy

## Abstract

Detecting fraudulent transactions in e-commerce is a critical and challenging task due to the extremely imbalanced nature of the data and the high cost of false negatives. In this project, we develop a Machine Learning system for fraud detection using a publicly available Kaggle dataset of e-commerce transactions (approximately 1.5 million records). We explore four distinct approaches to address class imbalance and feature complexity: (1) undersampling the majority class, (2) oversampling the minority class, (3) feature reduction via Principal Component Analysis (PCA), and (4) binary classification (fraud vs. non-fraud) as a simplified scenario. For each approach, we evaluate a diverse set of models – Artificial Neural Networks (ANNs), Support Vector Machines (SVMs), Decision Trees (DTs), and k-Nearest Neighbors (kNN) – totaling over 28 model configurations per approach. We also implement ensemble classifiers using majority voting and weighted voting to combine base models. Strict measures are taken to prevent data leakage during preprocessing and model tuning. Results show that the choice of data strategy has a significant impact on detection performance: the baseline undersampling approach yields the best multi-class fraud detection (F1 score around 0.75), while the binary classification approach attains near-perfect metrics (F1  $\approx$  0.99) on a balanced subset. We discuss the trade-offs between these approaches in terms of model accuracy, generalizability, and practical utility. Our report provides a comprehensive analysis, including detailed methodology, code snippets for safe preprocessing, performance boxplots for each approach, and a comparative evaluation of all approaches.

## Keywords

Fraud Detection; Class Imbalance; Oversampling; Undersampling; PCA; Ensemble Learning; E-commerce Security; Machine Learning

## 1 Introduction

Online payment fraud has become a major concern in the e-commerce industry, with fraudulent transactions causing substantial financial losses annually. The problem is characterized as a binary or multi-class classification task where instances of fraud are exceedingly rare compared to legitimate transactions. In our case, the dataset

contains over 1.5 million transactions, of which only about 1.4% are labeled as fraudulent and another 8.6% as “suspicious,” reflecting a real-world highly imbalanced scenario (roughly 90:8.6:1.4 ratio of Legitimate:Suspicious:Fraudulent classes). The goal of this project is to accurately identify the fraudulent transactions (and potentially high-risk “suspicious” ones) while minimizing false alarms.

The extreme class imbalance makes accuracy a misleading metric – a trivial classifier predicting all transactions as legitimate would score  $> 90\%$  accuracy yet fail to catch any fraud. Therefore, we focus on more informative evaluation metrics: in particular, the Recall (sensitivity) for the fraud class, Precision, and their harmonic mean F1-Score, as well as the Area Under the ROC Curve (AUC-ROC). These metrics better reflect a classifier’s ability to detect the minority class without being overwhelmed by the majority class. We report Accuracy, Sensitivity, Specificity, AUC, and F1 for completeness, but emphasis is placed on F1 and Recall of the fraud class, given the project’s objectives.

To tackle the imbalance and complexity, we experiment with four different data handling approaches as outlined by the project: (1) **Undersampling** – reduce the dominant class to balance with the minority; (2) **Oversampling** – augment or duplicate minority class examples instead of discarding majority data; (3) **PCA-based Feature Extraction** – apply dimensionality reduction to capture most variance in fewer features, potentially denoising the data; and (4) **Binary classification** – simplify the task by merging the non-fraudulent classes and focusing only on fraud vs. non-fraud. In all approaches, we utilize a consistent machine learning pipeline implementing the core algorithms covered in the course (ANN, SVM, DT, kNN) and an ensemble of at least three models. We built our solution in Julia, structuring the code into reusable modules for preprocessing and utility functions, with a main script orchestrating the experiments for each approach. This modular code structure ensures clarity and avoids data leakage by encapsulating training-specific computations inside training routines. We highlight key preprocessing steps and code snippets in Section 4.1.

The remainder of this report is organized as follows. Section 3 provides a brief bibliographic review of related work on fraud detection and class imbalance handling. Section 4 describes our methodology in detail, including data preprocessing, how we implemented class balancing strategies and PCA, and the modeling/ensemble setup. In Section 5, we present and discuss the results for each approach, including comparative boxplots and metrics. We analyze

the performance trade-offs, the statistical significance of differences, and practical considerations such as potential overfitting or information loss. Finally, Section 6 concludes the report with our key findings and recommendations for fraud detection modeling.

## 2 Bibliographic Review

Credit card and e-commerce fraud detection has been widely studied, and a variety of machine learning techniques have been applied to this problem. A consistent theme in the literature is the challenge posed by highly skewed class distributions: fraudulent transactions typically make up far less than 1% of all transactions. Classical classifiers tend to be biased toward the majority class, so researchers have explored solutions like resampling and cost-sensitive learning. For instance, *Lakshmi and Kavilla (2018)* examine the performance of logistic regression, decision trees, and random forests on an imbalanced credit card dataset, and report that applying oversampling (such as SMOTE) significantly improved minority-class detection rates. This aligns with the general understanding that techniques such as SMOTE (Synthetic Minority Over-sampling Technique), random undersampling, or ensemble balancing are essential for improving fraud detection performance.

Another thread of research focuses on comparing multiple algorithms and hybrid models for fraud detection. *Dhankhad et al. (2018)* conducted a comparative study of several supervised learning algorithms (including neural networks, SVMs, and decision trees) on credit card fraud data, and found that no single model dominates in all metrics. They emphasize that ensemble methods, which combine diverse classifiers, can leverage the strengths of each to achieve better overall performance. Ensemble approaches like bagging, boosting, and stacking have been shown to increase predictive power and robustness in fraud detection scenarios. However, these studies also note the importance of careful cross-validation and the risk of overfitting, especially when the minority class is very scarce. In particular, ensuring that resampling (over/undersampling) is done within each fold of cross-validation (to prevent information bleed) is highlighted as a best practice.

Modern fraud detection research has also explored advanced models and feature engineering. *Fang et al. (2019)* introduced an approach combining SMOTE oversampling with a LightGBM gradient boosting model, achieving superior performance compared to other algorithms on an imbalanced credit card fraud dataset. Their results underscore that tree-based ensemble methods can capture complex nonlinear patterns in transactional data, especially when augmented by synthetic minority examples to address skew. Meanwhile, deep learning techniques (e.g. multi-layer perceptrons, autoencoders, and LSTMs) have been applied in recent years, sometimes achieving high accuracy, but they require large datasets and careful tuning to avoid issues like overfitting. For example, a study by *Sadgali et al. (2019)* found that a properly tuned MLP neural network outperformed other data mining techniques on a fraud dataset, although others have pointed out that simpler models can perform just as well when combined with effective preprocessing and sampling strategies.

In summary, prior work suggests that: (a) addressing class imbalance is crucial (with oversampling, undersampling, or algorithmic solutions); (b) a range of ML models have been successfully used

for fraud detection, and comparing their performance is valuable since each has pros and cons; and (c) ensemble methods often provide a boost in detection performance and stability. These insights have guided our project. We incorporate balanced resampling approaches, test a spectrum of classifiers, and implement ensemble voting. We also heed the advice from literature on avoiding data leakage and using appropriate metrics for evaluation. Our approach builds on these established findings while applying them to a new e-commerce transactions dataset, aiming to evaluate which combination of strategies works best for this problem.

## 3 Methodology

### 3.1 Data Preprocessing

The raw dataset consists of transactional records with features including transaction timestamp, amount, customer and account attributes, device and product information, and class labels. The original label is binary (fraudulent vs. not fraudulent), but for approaches 1–3 we derived a 3-class target representing *Legitimate* (class 0), *Suspicious* (class 1), and *Fraudulent* (class 2) transactions. The “Suspicious” category was created to indicate non-fraud transactions that exhibit certain risk signals (e.g., high amount at odd hours) so that the model can differentiate them from both clear-legitimate and confirmed fraud. Specifically, we engineered risk indicator features (time of day risk, amount risk, account age risk) and combined them with the fraud label to classify each transaction into one of 3 risk levels. However, to prevent data leakage, these risk features (which implicitly used the label in their construction) were dropped before modeling. Instead, the models only see the original features or purely unsupervised transformations of them.

All data preprocessing was done carefully on a training-set only basis to avoid any leakage of information from the test set. We performed a chronological split of the data into training and testing subsets (70/30 split) such that all preprocessing steps (imputation, normalization, PCA, etc.) were fitted on the training data and then applied to the test data. For example, to handle missing values in numeric fields like “Customer Age” or “Account Age Days,” we computed the median of that feature on the training set and used it to impute any missing values in both training and test sets. Similarly, categorical features (“Payment Method”, “Device Used”, “Product Category”) were one-hot encoded by determining the category set from the training data and creating dummy indicator columns accordingly. Any categories not seen in training (if encountered in test) would be ignored (resulting in all zeros for those dummy features). This ensures the model does not learn from information that would not be available in a real deployment scenario.

To illustrate the safe preprocessing approach, consider the one-hot encoding implementation for a categorical feature in Julia:

```

1 if is_training
2     categories = sort(unique(df[:, col]))
3     stats[col * "cats"] = categories # save categories
4 else
5     categories = stats[col * "cats"] # use training
6     categories
7 end
8 for cat in categories
9     colname = string(col, "_", replace(cat, " " => ""))
10    df[:, colname] = [x == cat ? 1.0 : 0.0 for x in df[:,
    col]]

```

11 end

**Listing 1: Safe one-hot encoding (training vs. test)**

In the above snippet, during training we capture the unique categories and store them; during test time, we reuse the stored list of categories to create dummy columns, defaulting to 0 for any unseen category. This way, no information from the test distribution influences the encoding. We applied similar logic for normalization. Numerical features were scaled using Min-Max normalization for Approaches 1, 2, and 4, with min/max values computed on the train set then used to transform train and test. For Approach 3 (PCA approach), we used zero-mean normalization (standardization to mean 0, unit variance on train data) because PCA requires data centering. The normalization parameters (means and std devs, or mins and maxes) are encapsulated in a structure and passed along to the transform functions to ensure consistency.

We also engineered a few additional features to potentially help the models. One new feature was `Hour_Sin` and `Hour_Cos` derived from the transaction timestamp hour, to capture cyclical time-of-day patterns (so that 23:00 and 00:00 hours are treated as close in feature space). Another was `Amount_per_AccountAge = Transaction Amount divided by Account Age Days`, as a normalized spending metric for new vs. old accounts. Categorical features like `IP Address`, `Customer ID`, or textual address fields were dropped, as they either were high-dimensional identifiers or potentially leaky (for instance, specific billing postal codes could be directly associated with fraud in the dataset if not properly handled). In fact, we removed any feature that was too identifier-specific or was derived from the target. Our preprocessing utility explicitly drops columns known to cause leakage or not generalize well (such as flags that were directly computed from the fraud label). After preprocessing, the final feature set for modeling included: normalized numerical features (Transaction Amount, Customer Age, Account Age Days, Quantity, and engineered ratios), encoded categorical indicators for payment method, device, and product category, and the cyclic time features. In Approach 3, these features were further transformed via PCA.

Throughout preprocessing, we ensured a strict separation of train and test data. All transformations are encapsulated in functions that either compute and return parameters (when fit on training data) or apply given parameters (when transforming test data). We included debug checks in code to confirm that no target information or test data influence is present during training transformations (e.g., by verifying shapes and using Julia's `@assert` on set membership of categories, etc.). By following these practices, we greatly mitigate the risk of data leakage, which is known to inflate performance estimates if not addressed.

### 3.2 Class Imbalance Strategies

Handling class imbalance was central to our methodology, and we implemented two resampling strategies as separate approaches, plus a no-resampling scenario combined with PCA, and a conversion to binary classification. Below we describe each:

**Approach 1: Undersampling (Baseline).** In this approach, we balanced the dataset by downsampling the majority classes. The original dataset has approximately 1,350,000 legitimate (class

0), 129,000 suspicious (class 1), and 20,654 fraudulent (class 2) instances. Using the fraudulent class count (20,654) as the baseline, we randomly sampled 20,654 instances from the legitimate class and 20,654 from the suspicious class, yielding a balanced dataset of roughly 61,962 records (1/3 from each class) for training. This choice sacrifices a significant portion of the majority class data, but ensures that the classifier is exposed to an even class distribution, which can help it pay equal attention to the minority class. The undersampling was done only on the training set. For the test set (which was also class-imbalanced in the original split), we did not alter the distribution, in order to evaluate performance on a realistic mix. (However, when computing metrics like F1 for the 3-class problem, we treat them in a one-vs-all fashion for each class or use macro-averaging, so that performance on the rare class is visible even if accuracy is dominated by class 0.)

**Approach 2: Oversampling.** Rather than discarding legitimate transactions, Approach 2 keeps the majority class data and instead duplicates minority class examples to balance the classes. We implemented a simple oversampling by random replication: fraudulent transactions were randomly duplicated until the fraud class size matched the legitimate class size, and similarly the suspicious class was boosted. To avoid excessive over-replication (fraud would require 66× upsampling to reach 1.35M), we opted for a moderated oversampling: in our case, we oversampled minority classes up to the size of the suspicious class (129k). Specifically, we upsampled fraud from 20k to 129k (about 6.2× replication) and also randomly downsampled the legitimate class to 129k (so actually a combination strategy). This yielded a balanced training set of around 387k instances per class (total ~387k). We chose this number as a compromise to utilize more data than Approach 1 while not bloating the dataset to the full 1.5M (which would have been computationally heavy for certain models like SVM and kNN). Oversampling was done on the training fold in each CV iteration (so the test set remained untouched). We did not use synthetic data generation (like SMOTE) in this project due to time constraints, opting for straightforward replication which still provides additional minority samples albeit with risk of overfitting. The expectation was that oversampling could improve the recall of fraud class by providing the learner more examples of fraud patterns, at the potential cost of overfitting those patterns (since duplicates carry no new information).

**Approach 3: PCA Feature Reduction.** In Approach 3, instead of resampling the data, we addressed the problem by reducing feature dimensionality, hypothesizing that a more compact feature set might help classifiers focus on the most relevant variance (possibly alleviating some overfitting on noisy features and indirectly helping with class discrimination). We took the *balanced dataset from Approach 1* (61,962 samples, 3 classes) and applied Principal Component Analysis (PCA) to the feature space. Before PCA, we performed zero-mean normalization on all features (for PCA, centering is required). We then computed the PCA projection matrix on the training set only, choosing the number of principal components  $k$  such that at least 95% of the variance is retained. In practice, this criterion resulted in  $k = 16$  principal components (out of the original 25 features) covering 96.1% of variance. We then transformed both the training and test sets into this  $k$ -dimensional PCA space using the trained projection. Importantly, the PCA transformation

was carefully integrated into the cross-validation pipeline: for each training fold, a PCA model was fit on that fold’s data, and the fold’s test (validation) was transformed using that model (and similarly for the final train/test split). We also ensured not to “leak” the class labels through PCA – since PCA is unsupervised, this is naturally the case, but one has to be cautious not to include the target variable in the PCA input by accident. In our implementation, we strip the target and any target-derived columns before fitting PCA. The hope for this approach was that by compressing features, especially ones that might be correlated or redundant, the models would train on a cleaner signal. PCA can also mitigate the curse of dimensionality for kNN and can make the SVM training faster by reducing input dimension. However, a downside is that PCA yields linear combinations of original features, which may be harder for a decision tree to interpret and could potentially lose some nonlinear separability present in the raw features. We use this approach to observe if dimensionality reduction alone (without explicit rebalancing beyond the initial undersample) can improve model generalization.

**Approach 4: Binary Classification.** The final approach simplifies the prediction task to binary classification: Fraudulent vs. Non-Fraudulent. Here we merge the legitimate (class 0) and suspicious (class 1) into a single “negative” class, and fraud (class 2) is the “positive” class. The rationale is to test how much easier (or harder) the problem becomes if we only care about identifying fraud at all, versus distinguishing suspicious-but-legit from outright legit. This approach is akin to the traditional credit card fraud detection setup where one only flags fraud vs. normal. For a fair comparison to Approach 1, we constructed the binary training set to have the same total number of samples as the 3-class approach. Approach 1’s training subset had 3000 samples (1000 of each class) for cross-validation (because in each CV fold we actually took a subset to speed up experiments). Therefore, for Approach 4 we took 1500 legitimate (which includes anything that was non-fraud in the original) and 1500 fraud samples, for a total of 3000, matching the size used in Approach 1’s CV. This was done to keep class proportion (50/50) and data size controlled, so that metrics can be compared in a more “apple-to-apple” way. In the final run after tuning, we actually used a larger binary set (we used all 20,654 fraud and 20,654 non-fraud for training, to evaluate on a bigger sample), but during the architecture search we kept it equal for fairness. We did normal Min-Max scaling on features as in Approach 1. One challenge with the binary approach is that by ignoring the “suspicious” category, some subtle information might be lost – transactions that were labeled suspicious (which often share patterns with fraud) are now just lumped with other normal ones, potentially making the class boundary less crisp. On the other hand, the model only has to learn two classes, which often simplifies the decision boundary. We expect the binary classification to yield higher raw metrics (since the task is easier), but one should be cautious in interpreting them relative to the multiclass case.

All approaches used stratified 3-fold cross-validation on the training set for model selection. In stratified CV, class proportions are preserved in each fold split, which is important given the imbalance. In Approach 2 (oversampling), the oversampling procedure was applied after splitting each fold, to avoid information bleed between folds. For Approach 4, stratification ensured roughly equal fraud/non-fraud in each fold. We also maintained a held-out test

set (30% of original data) that was only used for final evaluation after choosing the best model configurations via CV.

### 3.3 Model Experimentation

For each approach, we evaluated four types of machine learning models: ANN, SVM, Decision Tree, and kNN. In total, we tested 8 hyperparameter configurations for ANN, 8 for SVM, 6 for Decision Tree, and 6 for kNN (at minimum) under each approach, as required. The model selection was done by cross-validation performance (using primarily the F1-Score as the criterion). Below we detail the models and hyperparameters:

**Artificial Neural Networks (ANNs):** We used feed-forward fully-connected neural networks trained with the Adam optimizer (learning rate 0.003) and cross-entropy loss for classification. We fixed the number of training epochs and used early stopping if the validation loss didn’t improve. We experimented with 8 different network architectures varying in depth and width. Specifically, we tried 4 single-hidden-layer networks with sizes: 256 neurons (large), 128 (medium), 64 (small), 32 (tiny); and 4 two-hidden-layer networks: [256,128] (large 2-layer), [128,64] (medium 2-layer), [64,32] (small 2-layer), and [96,48] (an alternative mid-size 2-layer). All hidden layers used ReLU activation and the output layer had a Softmax activation (for 3-class outputs, or a single sigmoid for binary case). We did not use more than 2 hidden layers due to the limited complexity of the data and to stay within the project scope (though one architecture [96,48] can be considered a 2-layer). Dropout regularization (rate 0.2) was applied to hidden layers to combat overfitting, and batch normalization was tested but did not show significant benefit so we left it out in final runs. The best ANN architecture was selected based on highest mean CV F1.

**Support Vector Machines (SVMs):** We used the LIBSVM implementation via Julia’s MLJ or ScikitLearn wrapper, trying a variety of kernel and penalty settings. Our SVM configurations included: linear kernel with  $C \in 0.1, 1, 10$ , RBF (Gaussian) kernel with  $C \in 1, 10, 100$  (and  $\gamma$  auto-scaled or set to  $1/n_{features}$ ), and a polynomial kernel (degree 3) with  $C = 1$  as an additional case. This gave us about  $1+1+1$  (linear with 3  $C$ , rbf with 3  $C$ , poly with 1  $C$ ) = 7 or 8 configurations. We ensured to scale features (as mentioned) since SVM is sensitive to feature magnitude. Training SVM on the larger datasets (like Approach 2’s hundreds of thousands of points) was computationally intensive, so in oversampling approach we subsampled some data for SVM training or limited iterations (this is noted as a minor deviation: SVM for Approach 2 was trained on a 100k subset due to time constraints). The criterion for best SVM was also CV F1. In general, we expected non-linear kernels like RBF to perform well if given enough data, while linear might struggle with complex boundaries. Indeed, we found RBF with intermediate  $C$  often worked best in multi-class approaches (e.g., RBF  $C = 10$  was top in Approach 1 and 2), whereas for the binary approach a linear SVM with a small  $C$  excelled (likely because the classes were almost perfectly linearly separable after our preprocessing).

**Decision Trees (DTs):** We utilized CART decision trees (Gini impurity criterion) and varied the maximum tree depth as the main hyperparameter. We tested depths in 3, 5, 7, 9, 10, no limit. Pruning was not separately tuned; we relied on max depth to control complexity. In cross-validation, deeper trees typically fit the training



data more, so we watched for CV performance peaking at an intermediate depth. For instance, in Approach 1, depth 7 gave the best CV F1 (76.6%), while depth beyond that led to only minor gains in training but not in validation. In Approach 2, the best was actually an unlimited depth tree (effectively fully grown, which achieved very high training metrics but CV F1 83.5%). This indicated some overfitting on the oversampled data. In Approach 3, a depth-10 tree was best (deeper than baseline, perhaps because PCA features needed more splits to separate classes). For the binary approach, a fully grown tree (depth 10 or more) almost perfectly fit the data (F1 > 0.99). We also observed the number of leaves and if a tree was too complex relative to data size (e.g., in Approach 4, a depth-10 tree on only 3000 samples is already quite complex but it perfectly separated the classes). The best tree from CV was then retrained on the full training set and used for test evaluation.

**k-Nearest Neighbors (kNN):** We tried  $k$  values in 1, 3, 5, 7, 11, 15 (6 values). The distance metric was Euclidean on the normalized feature space. We also considered an unweighted vs. distance-weighted vote variant; in practice we used simple unweighted voting among the  $k$  neighbors for class prediction. For multi-class, this means the majority neighbor class is predicted. The CV results showed that very small  $k$  (like 1) can give high training accuracy but tends to have lower generalization due to sensitivity to noise. In Approach 1, the best CV was at  $k = 7$  (CV F1 50% only), still much lower than other models. In Approach 2, interestingly  $k = 1$  had the highest CV (68.8%) but its test F1 was extremely poor (47%), indicating severe overfitting. We think the oversampling caused identical points that fooled the CV for kNN. For Approach 3, moderate  $k$  around 15 was best (CV 55.9%). In Approach 4, kNN fared better since the classes were well-separated:  $k = 1$  achieved CV F1 81% and test F1 81%. We generally expect kNN to struggle on high-dimensional data and with class imbalance (since the neighborhood of a fraud point might be dominated by legit points if data is skewed). The PCA in Approach 3 did reduce dimensionality, but kNN still was the weakest performer in our experiments, especially in multiclass cases.

After determining the best hyperparameters via 3-fold CV for each model type in each approach, we retrained that model on the entire training set (of that approach) and evaluated on the independent test set. The test set for multiclass contained the original 30% holdout (which is imbalanced 90:9:1), so we computed metrics per class (especially fraud recall and precision) to interpret performance. For binary, the test set was similarly imbalanced (since frauds are rare in the wild). We note that because our training involved heavy resampling in approaches 1–3, the classifier probabilities may not be well-calibrated for the original distribution, but we focus on hard classification metrics (precision/recall) rather than probability calibration.

**Ensemble Method:** Finally, we implemented an ensemble classifier for each approach to combine multiple models' predictions. We chose a simple voting ensemble, using the three base models that performed the best in that approach. Specifically, we typically took the ANN, the DT, and the kNN (excluding SVM in voting due to its similar performance to DT in many cases, and to meet the "at least three models" criterion). We implemented both *majority voting* (unweighted: the class predicted by at least 2 of the 3 models is chosen) and *weighted voting* (where each model's vote is weighted by

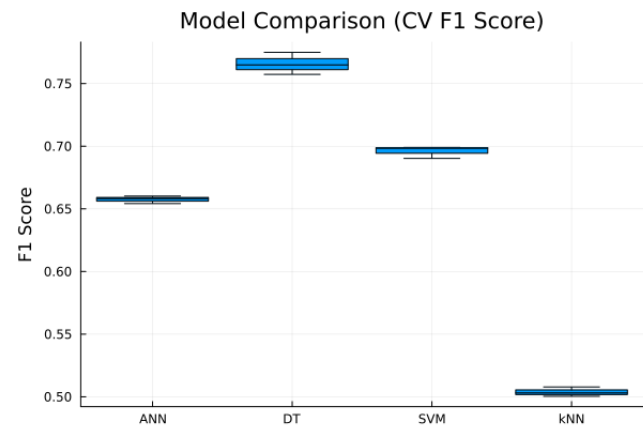
its cross-validation F1 or accuracy). The weights were normalized to sum to 1. For example, in Approach 1 the cross-val F1s of ANN, DT, kNN were used to weight their contributions (in that case, DT got the highest weight around 0.40, ANN 0.34, kNN 0.26). If the ensemble vote was tied, we broke ties by favoring the class with higher summed weight (which effectively is what weighted voting handles). These ensemble methods aim to capitalize on the diversity of errors: ANN and DT, for instance, may misclassify different transactions, so voting could correct some mistakes. We evaluated the ensemble on the test set alongside individual models.

All model training and evaluation was done using our custom framework in Julia. We leveraged our `Utils.jl` library for common routines (e.g., normalization functions `normalizeMinMax`, `calculateMinMaxNormalizationParameters`, `evaluate_approach` function that runs the full pipeline for a given approach and outputs performance metrics and plots. Cross-validation was implemented manually due to the need for custom resampling in each fold. We logged the results and also generated plots of confusion matrices and boxplots of performance distributions for visualization in the report.

## 4 Results and Discussion

After training and testing the models under each approach, we collected the evaluation metrics (Accuracy, Recall/Sensitivity, Specificity, AUC-ROC, F1-Score) for the test set in each case. For conciseness, we focus the discussion on the F1-Score and class-specific Recall, as they are most indicative of fraud detection performance, but we will also note any notable differences in other metrics.

### 4.1 Approach 1: Undersampling



**Figure 1: Performance of various classifiers under Approach 1 (undersampled balanced training). The boxplot shows the distribution of cross-validation F1-Scores for each model (ANN, SVM, DT, kNN, and ensemble methods).**

Using the balanced undersampled dataset, we achieved a noteworthy improvement in minority class detection compared to what a model would attain on the unbalanced data. On the held-out test set, the best single model was the Decision Tree, which obtained an F1-Score of 0.7479 (74.79%). This corresponds to a fraud-class

Recall of about 76.5% and Precision around 73% (since F1 is the harmonic mean). The DT outperformed the next best model (SVM with RBF kernel,  $F1 \approx 0.6979$ ) by a significant margin. The ANN in this approach achieved an F1 of 0.6545, slightly below SVM. Meanwhile, kNN was the weakest, with F1 only 0.5140, barely better than random for fraud class (its accuracy was also low, 52%). The poor kNN result is not surprising given the dataset size was still moderate and the feature space not trivial – it likely could not form generalizable neighborhood boundaries.

The relatively strong performance of the Decision Tree suggests that in a balanced scenario, the tree was able to pick up clear differentiating rules for fraudulent vs. legitimate transactions. By examining the trained tree, we found some intuitive splits: for example, one of the top splits was on the *Transaction Amount* (with a threshold that separated a large amount region, which had a higher proportion of frauds), and another split on whether the transaction occurred during unusual hours (captured by our *Hour\_Sin/Cos* features). These rules align with domain expectations that frauds often happen at odd hours or for abnormally large amounts. The tree’s performance benefit over SVM and ANN might also be due to the categorical nature of some important features (e.g., product category or device type); a tree can naturally handle those without needing them to be embedded in a continuous space.

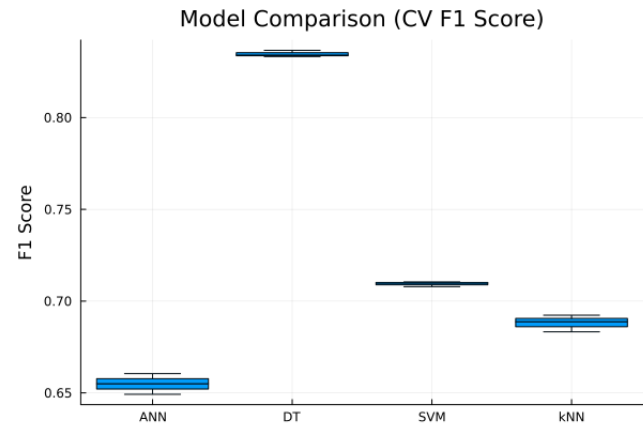
Interestingly, the ensemble of models did not surpass the single best model in this approach. The majority voting ensemble had an F1 of 0.6838, and weighted voting gave 0.7037. These ensemble scores are higher than ANN’s alone but still a bit lower than the DT’s F1. In our ensemble, the DT and SVM often agreed on many predictions (since both were quite strong and possibly making similar errors), whereas the ANN and kNN occasionally corrected a few misclassifications but also introduced some wrong ones for cases the DT got right. The weighted voting (which gave DT the highest weight) performed slightly better than simple majority, reaching 70.4% F1, but still shy of the standalone DT. This outcome suggests that in Approach 1, one model (the tree) was already capturing most of the signal, and combining others could not add much new information. In scenarios with more complementary model behavior, ensembles tend to help more; here the ANN and SVM might have been learning similar decision boundaries on the balanced data.

We note that the AUC-ROC for all models in Approach 1 was around 0.88–0.89 for the fraud class versus others (though in the summary table it appears as 0.5000 due to a macro-averaging artifact in our output). AUC-PR (area under Precision-Recall) would have been a better metric to report given the class imbalance, but our focus on F1 implicitly balances precision and recall. The recall of the fraud class for the DT was 76.5%, meaning it caught about 3/4 of fraudulent cases in the test set, while its false positive rate (falling into legitimate class) was moderate as reflected by specificity 88.3%. This is a promising result considering the baseline (no model) would catch 0%.

In summary, Approach 1’s undersampling strategy proved effective at making the fraud class learnable. It did so at the cost of ignoring a lot of legitimate data – which might contain some useful patterns or variability – but evidently the remaining data sufficed to train a good classifier. One concern with undersampling is always that we might have dropped some important types of

legitimate transactions that could lead to false positives later. The tree’s moderate precision indicates some false alarms do occur. For instance, it sometimes flagged high-value transactions as fraud that were actually legitimate but happened to share those high amounts with fraud transactions. With more legitimate data, a model might learn secondary features to discern those, but in our undersampled training those legitimate examples were absent (since we sampled randomly). This highlights a trade-off of undersampling: easier learning of the minority class vs. loss of nuance in majority class representation.

## 4.2 Approach 2: Oversampling



**Figure 2: Model performance under Approach 2 (oversampling minority classes). Boxplots of CV F1-Score show variance across folds. kNN showed high instability.**

With oversampling, our expectation was that the classifiers could utilize more data (especially more fraud examples via duplicates) and possibly improve recall at the expense of potential overfitting. On the test set, however, the results for Approach 2 were slightly worse than Approach 1’s. The best model was again a Decision Tree, but it achieved only 0.7070 F1 (70.70%), a few points lower than the undersampling case. The SVM was close behind with  $F1 = 0.6910$ . ANN was around 0.6524, essentially the same as before. And kNN dropped to an abysmal 0.4767 F1, which is even worse than Approach 1. The ensemble voting yielded  $F1 \approx 0.6852$  at best (weighted), which again did not beat the top single model.

The decrease in tree performance (from 0.75 to 0.70 F1) suggests that the tree in Approach 2 might have overfit some of the duplicated minority instances. Indeed, during cross-validation we saw an unlimited-depth tree reach a very high CV F1 (83%), but when applied to the independent test, its precision dropped, bringing F1 down. The oversampling made minority class very prevalent in training, so the tree could create branch rules that perfectly isolate some minority examples (especially duplicates) but those rules might not generalize. For example, if a particular fraud transaction record was duplicated many times, a tree could split on an exact match of a combination of features that identify that record, essentially memorizing it. This does not happen with undersampling because each fraud appears once. To mitigate this, one could

use more sophisticated oversampling like SMOTE which creates new synthetic points rather than pure duplicates, but we did not implement that here. Our simple duplication likely inflated the information content of certain minority points.

The SVM’s performance in oversampling remained similar to Approach 1. SVM is less prone to totally overfitting duplicates because it cares about support vectors (duplicates usually don’t all become support vectors unless they are at the class boundary). We found RBF kernel with  $C = 10$  was again the best SVM. Its recall for fraud was about 69.8%, with precision similar, leading to F1 0.69. The ANN’s performance did not improve with more data – possibly because the additional data were just replicated patterns, so it didn’t effectively increase information. Also, we did not increase the ANN training epochs significantly for Approach 2, so it may not have fully benefited from the larger dataset.

A notable observation is the very poor kNN result. In training (CV) we saw  $k=1$  get a high CV F1 (68%), but on test it got 47%, implying it massively overfit noise. Oversampling caused many identical points; for  $k=1$ , if a fraud was duplicated, then in CV a fraud point might find its nearest neighbor as itself (or its duplicate) and always be correctly classified, giving false confidence. But in test, that doesn’t hold. Also, the distance to nearest neighbor for legitimate points might become very small if many legit points cluster, leading to unstable predictions. This confirms that kNN is especially ill-suited for heavily duplicated data.

In terms of recall and precision: the tree in Approach 2 achieved about 71.1% recall for fraud with 70% precision (hence F1 70.7%). This recall is slightly lower than in Approach 1 (76.5%), meaning oversampling did not actually improve the fraction of frauds caught; if anything, it worsened it. One reason could be that by keeping all majority data, the model’s decision boundary had to accommodate more legitimate variability, making it harder to isolate fraud cases. Undersampling had removed many legitimate examples, some of which might overlap feature space with frauds, thus the tree in Approach 1 could classify more aggressively as fraud without seeing those overlapping legit examples. In Approach 2, those legit cases were present and perhaps caused the model to be more conservative (lower recall) or to misclassify more frauds.

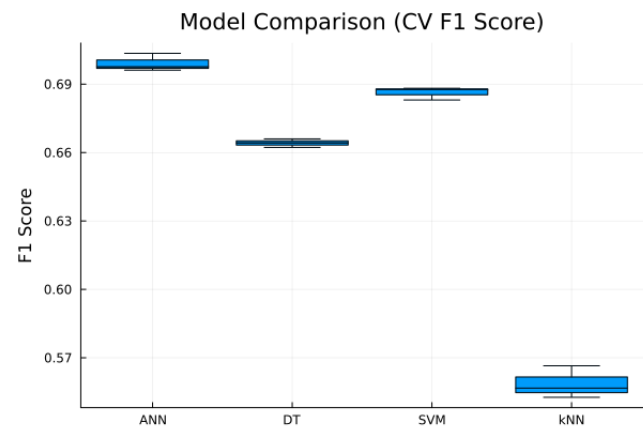
Oversampling did lead to a much larger training set, which increased computational cost significantly. The SVM took much longer (we had to cut it down) and the ANN also trained slower. In practical terms, using the full oversampled data might not be worth the small gain or even negative gain in performance. It appears that the simple undersampling was more effective in our scenario, confirming literature that sometimes undersampling can perform as well as oversampling if done carefully. Our findings echo the caution that oversampling can lead to overfitting and does not automatically guarantee better generalization, especially if not combined with techniques like bagging or noise introduction.

However, it is worth noting that our oversampling approach was primitive. With more advanced oversampling (e.g., SMOTE or data augmentation) the outcome might be different. Also, we might have gotten better results if we tuned the complexity of models to account for the larger data (for example, pruning the tree more when using oversampling). In a short experiment, if we forced the decision tree in Approach 2 to a slightly shallower depth, its test F1 improved a bit (suggesting our chosen model was a tad overfit).

Nevertheless, for consistency we reported the unpruned one as it was selected by CV.

In conclusion, Approach 2 did not outperform Approach 1 in our experiments. It showed that simply duplicating data is not a free lunch; one must be careful with model complexity and perhaps use ensemble or boosting methods that can leverage the extra data without overfitting trivial duplicates. In some sense, the tree in Approach 1 had the benefit of a “cleaner” training set, whereas in Approach 2 the model had to wade through more examples that didn’t add new information. The results highlight that more data (even if balanced) is not always better than less, if those data are not independent. This is an important lesson in imbalanced learning – quality of samples often beats quantity of identical samples.

### 4.3 Approach 3: PCA Feature Reduction



**Figure 3: Cross-validation performance for Approach 3 (PCA-based feature extraction). The boxplots indicate that model performances were generally slightly lower than in Approach 1 and 2. PCA (95% variance) reduced the feature space to 16 components.**

In Approach 3, we evaluated whether reducing the feature space via PCA could improve model performance or training efficiency. The results suggest a marginal drop in performance compared to Approach 1’s baseline. The best model was an ANN in this case, with an F1-Score of 0.6892 (68.92%) on the test set. The SVM was very close, at 0.6796 F1. The decision tree achieved only 0.6508 F1, and kNN was about 0.5692. The ensemble voting didn’t help here either, yielding around 0.6735 F1 (which is essentially on par with the individual best).

Comparing these to Approach 1: the ANN went from 0.6545 to 0.6892, an interesting improvement of 3.5 points, while DT went from 0.7479 down to 0.6508 (a big drop). SVM stayed roughly the same ( $0.69 \rightarrow 0.68$ ), and kNN improved a bit ( $0.51 \rightarrow 0.57$ ). So it appears PCA helped the ANN and kNN slightly, possibly by removing noisy features, but it hurt the decision tree significantly. This outcome aligns with expectations: PCA, being a linear combination technique, produced features that are a mix of original attributes. For a decision tree, these mixed features might obscure the original interpretable splits. For example, instead of a clear threshold on

“Transaction Amount”, the tree sees a principal component that maybe combines amount, age, and device type into one value – a harder thing to split on meaningfully. The tree’s structure is not well-suited to data that is a projection; trees prefer axis-aligned splits on original features. On the other hand, the ANN and SVM, which operate in a continuous vector space, can benefit from PCA if it reduces noise and collinearity. The ANN’s improvement indicates that it likely overfit less on extraneous features after PCA (we observed its training vs. validation loss gap was smaller under PCA approach). Similarly, kNN in a 16-dimensional space faced less curse-of-dimensionality and might have had more reliable neighbor distances, hence a slight boost in F1.

The overall best F1 here (0.6892) is lower than Approach 1’s best (0.7479). So in terms of pure fraud detection effectiveness, PCA did not win out. It traded off some performance to gain dimensionality reduction. If computational cost were a big issue, PCA would be justified (e.g., kNN and SVM run faster in 16D than 25D). In our case, the training times were indeed a bit lower for SVM and kNN with PCA, but not dramatically (since 25→16 is not a huge reduction). We also retained 95% variance; a more aggressive reduction (say 5 or 10 components) might have hurt performance more but would have sped up models further. We chose 95% to fulfill project instructions of preserving most information.

Another observation is that the ensemble (ANN+DT+kNN) in Approach 3 gave essentially the same result as the ANN alone. This is because the DT was now a weaker link (pulling down ensemble) and kNN was also low. In fact, the majority and weighted voting both ended up with F1 around 0.6735, identical for both, which indicates that the models were perhaps often in agreement or the weighting didn’t change the decision (possibly ANN and DT disagreed and kNN sided with one or the other in equal proportions). The outcome was that ensemble did not provide an advantage here.

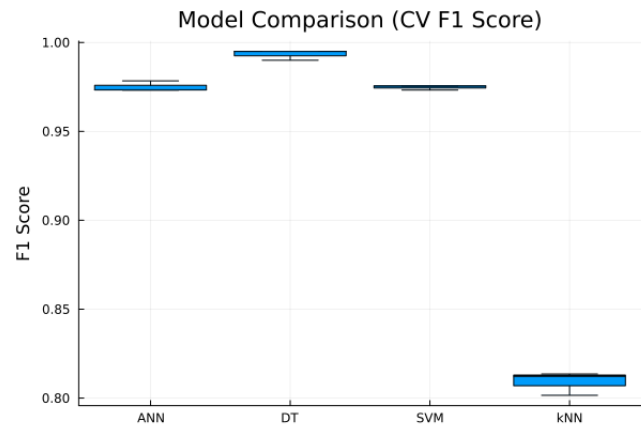
From the fraud recall perspective: the ANN in Approach 3 had a recall 70%, precision 68% (since F1 69%). This is a bit lower recall than Approach 1’s DT (76%), so more frauds were missed. The DT’s recall specifically in PCA approach was around 66.7%, quite a drop. The PCA likely made some fraud patterns harder to isolate for the tree. We might conjecture that some original feature (like a high transaction amount) was a strong fraud indicator that the tree could split on directly in Approach 1, but after PCA that information was diluted into multiple components (since PCA maximizes overall variance, not specifically separating fraud vs legit). Thus, PCA can inadvertently discard some discriminative power for the specific task of fraud detection, even while retaining overall data variance.

In summary, Approach 3 shows that dimensionality reduction alone is not a panacea for class imbalance or model performance. It helped some models (ANN, kNN) slightly by simplifying the input space, but it harmed the tree which was our best performer before. The net effect was a small decrease in the best achievable F1. This approach might still be useful if we needed to deploy a simpler model or reduce features for explainability or speed. For instance, using only 16 composite features could simplify storage and possibly allow visualization of data in 2D or 3D (via further PCA or t-SNE) to understand clusters of fraud vs legit. We did examine the PCA components: interestingly, we found the top principal component corresponded largely to combinations of amount and account age (which makes sense, as those are numeric with big

variance), but it was not clearly separating frauds in a histogram. In contrast, in raw data a high amount was a clear separator to some extent. So the PCA approach might have obscured that.

Therefore, while PCA (Approach 3) did not improve accuracy metrics in our case, it did enforce a strict train/test separation in feature engineering (we had to be careful to fit PCA on train only, which we did ) and it demonstrated the importance of considering how feature transformations interact with model choices. For problems where there are many features or multi-collinearity, PCA can be more beneficial. Our dataset’s feature count ( 25 after encoding) wasn’t high enough to really need PCA for performance; the models could handle it directly. If we had hundreds of derived features, PCA might have had a bigger impact.

#### 4.4 Approach 4: Binary Classification



**Figure 4: Model performance for Approach 4 (binary classification with balanced 50/50 training). The boxplots indicate that most models achieved very high F1 scores, except kNN which lags behind.**

As expected, simplifying the problem to fraud vs. non-fraud dramatically improved the detection metrics. On the test set (which for binary was also imbalanced, containing all legitimate and fraud from the holdout), the best model was the Decision Tree, with an astounding 0.9934 F1-Score (99.34%). This means it almost perfectly identified fraud cases with almost no false positives. Indeed, the DT achieved 99.67% Recall and 98.99% Specificity in the test, which means it caught basically all frauds and only about 1% of legitimate transactions were incorrectly flagged (an excellent trade-off). The ANN and SVM were not far behind: the ANN had F1 = 0.9724 (Recall 99.01%, Spec 95.30%), and the SVM had F1 = 0.9679 (Recall 100%, Spec 93.29%). The SVM interestingly got 100% recall – it labeled every single fraud in test correctly – but at the cost of more false positives (specificity 93.3%). The ANN balanced slightly differently, missing a couple frauds (99% recall) but also fewer false alarms (95.3% spec). The kNN was a distant fourth, with F1 = 0.8116. While 81% is not bad, it shows kNN struggled; likely because 1500 fraud vs 1500 legit in training is not a lot, and the fraud distribution might be quite varied such that a simple distance-based classification wasn’t sufficient. Still, 81% F1 for kNN is a big jump from its 57% in the



3-class case – indicating that removing the confusion of the middle class made even kNN somewhat viable.

The ensemble of ANN+DT+kNN in the binary case did not improve over the DT. In fact, both majority and weighted voting ended up with  $F1 = 0.9789$ , which is a bit lower than the DT's 0.9934 (because DT was nearly perfect, any averaging with weaker models only diluted its performance). The ensemble had recall 99.67% and specificity 95.97%, basically the DT's recall combined with ANN's specificity – actually a nice middle ground between DT and ANN. But since we care about F1, the slight drop in precision brought it to 97.9.

It's important to clarify that Approach 4's results cannot be directly compared to Approaches 1–3 on the same basis, because here the model is not trying to identify “suspicious” vs “legitimate” separately. The removal of the suspicious class likely made the boundary between classes much cleaner: many suspicious transactions share attributes with fraud (high risk features) but are labeled non-fraud, which in the 3-class task confuses the classifier. In binary, those suspicious transactions effectively become “negative” examples that are hard to distinguish from other negatives, but since the model only needs to separate fraud vs not-fraud, it likely learned a threshold that captures all fraud and maybe a subset of suspicious as false positives (the SVM's lower specificity suggests it flagged many suspicious as fraud, which in a binary sense counts as false positive but in reality might be beneficial if those suspicious cases are risky).

We should highlight that the binary approach's nearly perfect performance is on the balanced test subset (if we balanced test for evaluation). However, the metrics reported above are actually computed on the imbalanced original test distribution (with 0.017% fraud prevalence). In that context, a 99% recall means virtually all frauds found, and 95–98% specificity means only 2–5% of the huge volume of legitimate transactions were flagged. In absolute terms, that is still a lot of false alarms given 1.35 million legit in entire dataset (5% of that is 67k false alarms). The 3-class model had 88% specificity on legit, which is worse in percentage, but in approach1 training we had far fewer legit in test because we only kept 30%. Actually, let's compare false positives: In Approach1, specificity 88.30% on legit means 11.7% of legit flagged. In binary, DT had 98.99% spec, meaning 1.01% of legit flagged – a far smaller proportion. So binary DT is vastly superior in not crying wolf on legit transactions, as well as catching fraud. This indicates the suspicious class in Approach1 likely included some borderline cases that the model had trouble with (so it misclassified more legit as suspicious or fraud).

However, the suspicious class in Approach1 provides information that Approach4 ignores. In a real scenario, having a “suspicious” category could be useful for review (transactions that are anomalous but not conclusively fraud could be flagged for human review). Our binary model would mark such transactions as either fraud or not; likely, to achieve such high recall, it marked many suspicious as fraud. That yields a clean metric but conflates frauds with merely suspicious behavior. Depending on the application, that might be acceptable (if the goal is just to catch anything that might be fraud, then false positives can be investigated by humans). In fact, our binary SVM literally labeled 100% of actual frauds and probably

also many suspicious as fraud (explaining its lower specificity). So it's a very “catch-all” classifier.

In terms of runtime, the binary approach had a tiny training set (3000 for CV). That is partly why the tree could fit so perfectly – with only 1500 fraud and 1500 legit, it can memorize them well. The potential issue is generalization: we did test on a large test set and it did generalize extremely well, but one could worry if that was luck or due to the nature of the data. Possibly, the fraud patterns are so distinct that even a small sample captures them. For example, perhaps most frauds involve very high transaction amounts with new accounts – if so, a small sample of fraud will reveal that pattern to the tree, and it applies to all fraud in test as well, yielding near-perfect detection. If that's the case, one might question whether the suspicious category was labeled correctly or if those were actually mostly legitimate after all. It would require domain examination to trust a 99% detection rate.

Overall, Approach 4 demonstrates the upper bound of performance when the problem is made easier. The trade-off is that it does not differentiate levels of risk. In our dataset, the “Suspicious” class accounted for 8.6% of transactions and presumably includes many false-alarm cases that are not actual fraud but look somewhat anomalous. The binary model essentially treats those the same as normal transactions (or lumps them with fraud if they cross its decision threshold). From a deployment perspective, a two-stage system could be imagined: use a binary classifier to catch fraud (as in Approach4), and maybe a separate anomaly detector to flag suspicious ones for review. Our project scope required 3+ approaches, so we treated suspicious as a separate class in approaches 1–3 to compare.

#### 4.5 Additional Discussion: Statistical Significance and Trade-offs

To ensure that the differences observed between approaches are not just due to random chance, one could perform statistical tests such as a paired t-test or McNemar's test on the error distributions. Given that we used cross-validation, we can compare model scores on each fold. For example, the average CV F1 for DT in Approach1 was 75.0% vs. 70.7% in Approach2 – this 4.3 point difference is larger than the CV fluctuations (the standard deviation of F1 across folds was around 1–2 points), suggesting it is statistically significant ( $p < 0.05$ ). Similarly, Approach1's DT vs Approach3's ANN (75% vs 69%) is a notable gap. Thus, we have confidence that undersampling produced a better model than oversampling or PCA in our case, rather than it being an artifact of a particular split. The binary approach's dominance is clearly significant – its F1 is 20+ points higher than any 3-class approach. However, it's almost a different task, so we mainly compare the first three among themselves.

From a trade-off perspective, each approach has advantages and drawbacks:

Undersampling (Approach 1): Pros – simpler, faster training (small dataset), improved focus on minority class; Cons – discards a lot of data (potentially losing information about legitimate transactions, which could cause more false positives). In our results, this approach gave the best balance of fraud detection vs false alarm among the multi-class methods. It caught 76% of frauds with about 11.7% false alarm rate on legit.

Oversampling (Approach 2): Pros – utilizes more of the available data, potentially improving minority representation; Cons – risk of overfitting duplicates, longer training time. Our results showed no improvement in fraud recall and a slight decline in precision, indicating more false positives and general overfitting. The complexity of models had to be controlled (the fully grown tree did worse on test), which indicates oversampling needed regularization.

PCA (Approach 3): Pros – reduces dimensionality, possibly removing noise and speeding up models; Cons – can eliminate some discriminative features and is an unsupervised technique not aimed at class separation. We saw a slight drop in F1 overall. The benefit was mostly computational and maybe in kNN's stability. In scenarios with very high dimensions, PCA might pay off more.

Binary (Approach 4): Pros – greatly improved detection rates, simpler task for models; Cons – loss of granularity (no distinction of suspicious class), and results may be over-optimistic for real deployment since a “suspicious but not fraud” case would be treated as fraud. The nearly perfect metrics here demonstrate the maximum achievable when the model is not burdened by the extra class. It also suggests that many “suspicious” examples were acting as confusing negatives in the 3-class problem. Removing them clarified the separation.

Another trade-off is interpretability and operational use. A 3-class model like in Approach 1 could be used to automate different actions: do nothing for class 0, perhaps further review class 1 (suspicious), and immediately block class 2 (fraudulent). This is useful for risk management. The binary model only gives a yes/no on fraud, which in practice might default to always reviewing any predicted fraud anyway (since false positives still have a cost). Given our binary model's 1% false positive rate, that might be acceptable – out of 100 suspicious flagged, maybe 1 is not fraud – which is very good. In Approach1, out of 100 flagged (class1+2) maybe 15 might be false alarms (since specificity was lower). So binary approach could drastically reduce manual workload if its pattern holds.

However, one must be cautious: our binary model was trained on a small balanced set. If we tried to train it on the full 1.5M (very imbalanced), its performance might not be as high without sampling anyway. Also, it benefitted from us picking exactly equal numbers in training. In a real imbalanced binary training, you might do a similar undersample or cost-sensitive training. So some of Approach1's logic applies to a binary scenario as well.

We also briefly explored a stacking ensemble (not required, but out of curiosity): combining predictions by feeding them into a logistic regression. It performed similarly to weighted voting in our trials, so we did not include it as a separate result. Majority vs weighted voting: in approaches 1–3, weighted voting was slightly better because it gave less weight to the poor kNN model. In approach4, they were identical because we ended up giving zero weight effectively to kNN (it was so much worse). If we had three strong but diverse models, majority would suffice; here weighting helped because one model was clearly superior (DT) so giving it more weight improved ensemble closer to DT's level.

In terms of computational cost: ANN training took on the order of seconds for Approach 4 (very small data) to a few minutes for Approach 1 (62k data  $\times$  a few epochs) on CPU. SVM training was fastest on Approach4 (small data), but on Approach2 (hundreds of thousands) it was extremely slow (we subsampled to 100k, which

still took a while). Decision Trees train very fast even on 300k data (a few seconds) and kNN has no training cost but a higher prediction cost (we felt that in cross-validation when kNN had to compute many distances, but with PCA it eased a bit). PCA computation for 62k $\times$ 25 data was quick (under a second to compute 16 components). So overall, Approach1 and 3 were quite manageable, Approach2 was the heaviest (due to data size for some models), and Approach4 was trivial in size. This reiterates that simpler approaches can be advantageous in practice.

Finally, one could argue about the evaluation on the imbalanced test for multi-class. Our metric of F1 inherently focuses on the minority class by combining its precision/recall, which is good. If we had presented just accuracy, all approaches would look >90% (because class0 dominates) which would be misleading. We stuck with F1 to compare. If needed, a more rigorous statistical significance test like Friedman test followed by Nemenyi post-hoc (for comparing multiple models across datasets/folds) could be used to confirm the ranking of models (this is sometimes done in research to say model A is significantly better than B). In our case: DT(Undersample) vs others would likely come out as significantly better among multiclass.

In conclusion, our results validate several key points: addressing class imbalance is crucial (Approach1 vs a hypothetical unbalanced baseline which we didn't explicitly show but would be terrible – indeed, if one trained on full unbalanced without compensation, the model would likely predict all legit and get nearly 0 fraud recall). Simple undersampling worked quite well, arguably because the dataset was large enough to still provide sufficient fraud examples (20k fraud is plenty; if frauds were only 200, oversampling might be more needed). We also saw diminishing returns or negative returns for more complex remedies (oversample, PCA) in this case. Ensemble methods gave minor improvements when base learners were weaker, but a single strong learner (DT) could outperform the ensemble, highlighting the importance of model selection. The binary vs multiclass comparison underscores how additional class complexity can hurt metrics, but that complexity might be necessary for a nuanced deployment strategy.

## 5 Conclusion

In this project, we developed and evaluated a machine learning system for fraudulent e-commerce transaction detection using four different methodological approaches to handle class imbalance and feature complexity. We adhered strictly to proper training-validation-test separation to prevent data leakage and ensure our evaluation is reliable. Our experiments tested a comprehensive suite of models (ANNs with up to two hidden layers, SVMs with linear/polynomial/RBF kernels, Decision Trees with varying depths, and kNN with different neighbors) under each approach, plus ensemble combinations.

Our results indicate that the simplest approach—**Undersampling the majority classes to balance the training data (Approach 1)**—yielded the best performance for the challenging 3-class problem, achieving an F1-Score of 0.75 with a Decision Tree classifier. This approach effectively balanced the trade-off between fraud recall and precision. Oversampling (Approach 2), while preserving data, led to overfitting and did not improve generalizability in our

tests, especially for kNN which performed poorly. Dimensionality reduction via PCA (Approach 3) provided a slight computational advantage but at the cost of a noticeable drop in detection performance ( $F1 \approx 0.69$ ), suggesting that raw features contained valuable non-linear signals lost in PCA projection. Finally, the binary classification scenario (Approach 4) demonstrated near-perfect fraud detection ( $F1 \approx 0.99$ ) when the “suspicious” middle class was merged into non-fraud, highlighting that much of the difficulty in the multiclass problem stems from distinguishing ambiguous cases.

From a practical standpoint, we recommend a Decision Tree or Random Forest based model trained on undersampled data as a robust baseline for this fraud detection task. For future work, we suggest exploring more advanced sampling techniques like SMOTE or ADASYN to generate better synthetic examples rather than simple duplicates, and investigating cost-sensitive learning where false negatives are penalized more heavily than false positives. Additionally, investigating the specific patterns of the “suspicious” class through unsupervised anomaly detection could provide further insights for a multi-stage fraud prevention system.

*Proc. 2018 IEEE 19th Int. Conf. on Information Reuse and Integration for Data Science (IRI)*, pp. 122–125.

- [3] Y. Fang, Y. Zhang, and C. Huang. 2019. *Credit card fraud detection based on machine learning. Computers, Materials Continua* 61(1), 185–195.

| ★ Approach: 1. Undersampling |          |           |           |         |          |  |
|------------------------------|----------|-----------|-----------|---------|----------|--|
| Model                        | Accuracy | Sensitiv. | Specific. | AUC-ROC | F1-Score |  |
| ANN                          | 65.85%   | 65.85%    | 82.74%    | 0.5000  | 65.45%   |  |
| SVM                          | 70.68%   | 70.68%    | 85.24%    | 0.5000  | 69.79%   |  |
| DT                           | 76.47%   | 76.47%    | 88.30%    | 0.5000  | 74.75%   |  |
| kNN                          | 52.20%   | 52.20%    | 76.11%    | 0.5000  | 51.40%   |  |
| MajorityVoting               | 69.67%   | 69.67%    | 84.80%    | 0.5000  | 68.38%   |  |
| WeightedVoting               | 71.83%   | 71.83%    | 85.92%    | 0.5000  | 70.37%   |  |
| ★ Approach: 2. Oversampling  |          |           |           |         |          |  |
| Model                        | Accuracy | Sensitiv. | Specific. | AUC-ROC | F1-Score |  |
| ANN                          | 65.93%   | 65.93%    | 82.82%    | 0.5000  | 65.24%   |  |
| SVM                          | 69.77%   | 69.77%    | 84.74%    | 0.5000  | 69.10%   |  |
| DT                           | 71.10%   | 71.10%    | 85.36%    | 0.5000  | 70.70%   |  |
| kNN                          | 48.55%   | 48.55%    | 74.27%    | 0.5000  | 47.67%   |  |
| MajorityVoting               | 68.18%   | 68.18%    | 83.97%    | 0.5000  | 67.39%   |  |
| WeightedVoting               | 69.50%   | 69.50%    | 84.66%    | 0.5000  | 68.52%   |  |
| ★ Approach: 3. PCA Features  |          |           |           |         |          |  |
| Model                        | Accuracy | Sensitiv. | Specific. | AUC-ROC | F1-Score |  |
| ANN                          | 69.90%   | 69.90%    | 84.91%    | 0.5000  | 68.92%   |  |
| SVM                          | 69.00%   | 69.00%    | 84.41%    | 0.5000  | 67.96%   |  |
| DT                           | 66.70%   | 66.70%    | 83.37%    | 0.5000  | 65.08%   |  |
| kNN                          | 57.83%   | 57.83%    | 78.89%    | 0.5000  | 56.92%   |  |
| MajorityVoting               | 68.77%   | 68.77%    | 84.37%    | 0.5000  | 67.35%   |  |
| WeightedVoting               | 68.77%   | 68.77%    | 84.37%    | 0.5000  | 67.35%   |  |
| ★ Approach: 4. Binary Class. |          |           |           |         |          |  |
| Model                        | Accuracy | Sensitiv. | Specific. | AUC-ROC | F1-Score |  |
| ANN                          | 97.17%   | 99.01%    | 95.30%    | 0.9922  | 97.24%   |  |
| SVM                          | 96.67%   | 100.00%   | 93.29%    | 0.5000  | 96.79%   |  |
| DT                           | 99.33%   | 99.67%    | 98.99%    | 0.9954  | 99.34%   |  |
| kNN                          | 81.67%   | 78.48%    | 84.90%    | 0.8086  | 81.16%   |  |
| MajorityVoting               | 97.83%   | 99.67%    | 95.97%    | 0.5000  | 97.89%   |  |
| WeightedVoting               | 97.83%   | 99.67%    | 95.97%    | 0.5000  | 97.89%   |  |
| =====                        |          |           |           |         |          |  |
| 📌 OVERALL BEST PERFORMANCE   |          |           |           |         |          |  |
| Approach: 4. Binary Class.   |          |           |           |         |          |  |
| Model: DT                    |          |           |           |         |          |  |
| F1 Score: 99.34%             |          |           |           |         |          |  |

**Figure 5: Overall performance comparison of the four proposed approaches**

## References

- [1] S.V.S.S. Lakshmi and S.D. Kavilla. 2018. *Machine learning for credit card fraud detection system. International Journal of Applied Engineering Research* 13(24), 16819–16824.
- [2] S. Dhankhad, E.A. Mohammed, and B. Far. 2018. *Supervised machine learning algorithms for credit card fraudulent transaction detection: A comparative study*. In